

Fuzz/Test go-cpython

GSoC Student Application & Proposal

Linux Foundation GSOC Program 2026

Student Name	Abhishrestha Tiwari
Major	Information Technology
University	Madan Mohan Malaviya University of Technology
Github Profile	@ https://github.com/abhishrestha
Linkedin	http://www.linkedin.com/in/abhishrestha-tiwari
Email	tiwariabhishrestha23@gmail.com
Mentor	Till Kamppeter, Jiongchi Yu, Alexander Pevzner

This application contains:

- [About Me](#)
- [GSOC Background](#)
- [Experience in OpenPrinting](#)
- [Project Proposal](#)
 - [Abstract](#)
 - [Objectives](#)
 - [Methodology](#)
- [Tentative Timeline and Deliverables](#)
- [Availability](#)
- [Post GSOC](#)

About Me

I am a final year undergraduate pursuing my **Bachelor's in Information Technology** at **Madan Mohan Malaviya University of Technology (MMMUT)**.

I possess proficiency in **C, C++, Java, JavaScript, Go, and Python**, with a strong focus on systems programming, low-level debugging and open-source development..

Development Environment

- **Hardware:** Macbook M4, 16 GB RAM, 512 SSD
- **Operating System:** Ubuntu 24.04 (via Lima VM on ARM64), macOS
- **Shell:** Bourne Shell (Bash)
- **IDE:** Visual Studio Code (Remote-SSH to Lima VM)

GSoC Background

I have a strong interest in systems-level software, security testing, and open-source infrastructure. After evaluating GSoC 2026 organizations, I found that OpenPrinting under the Linux Foundation aligns best with my expertise — particularly in **CPython** internals, **Go/C interop**, and **fuzz testing**.

I am exclusively applying to the Linux Foundation for GSoC 2026. Having already contributed a merged bug fix to the **go-mfp cpython** package, I bring direct, hands-on familiarity with the exact codebase this project targets.

Experience in OpenPrinting

Fuzzing Background — cups-browsed

As part of my OpenPrinting exploration, I developed a fuzzing harness for **cups-browsed** and worked on integrating it with **OSS-Fuzz** infrastructure. This gave me hands-on experience with:

- **AFL++** mutation-based fuzzing and seed corpus design
- **LibFuzzer** harness development for C-based projects
- **AddressSanitizer** and **UBSan** for crash detection and undefined behavior
- **Crash triage**, minimization, and reproduction workflows

go-mfp CPython Binding — Bug Fix (Merged PR)

While running *go test ./...* inside a Lima Ubuntu 24.04 **ARM64** VM, I discovered that two tests — *TestObjectFromPython* and *TestKyoceraESCLScannerCapabilities* — hung indefinitely. I investigated using **gdb**:

- Compiled the test binary and set a breakpoint on *py_interp_close*
- Stepped through the C function line by line, confirming *PyThreadState_Swap* completed successfully on both calls
- Identified the hang inside *Py_EndInterpreter* → *wait_for_thread_shutdown()* → *threading._shutdown()* → *sem_wait()*

I researched the CPython issue tracker and found **CPython #122517** (Deadlock in *Py_EndInterpreter*) and **CPython #87135** (Hang non-main threads during GIL finalization), confirming this as a known CPython bug on ARM64.

Following guidance from Alexander Pevzner, I implemented the fix by replacing *threading._shutdown* with a no-op via *pyGate.eval()* before *Py_EndInterpreter* is called. Verified on **ARM64 (Python 3.12)** and by Alexander on **Fedora/x86 across Python 3.9–3.15**. The PR was merged.

(<https://github.com/OpenPrinting/go-mfp/pull/21>)

Deep Familiarity with the CPython Binding Codebase

Through this contribution I developed deep familiarity with the **cpython** package internals:

- Sub-interpreter lifecycle: *py_new_interp*, *py_interp_close*, *pyGate* as mutex
- Read and understood the full implementation of *cpython.c*, *gate.go*, *python.go*, and *object_test.go* — including the C/Go boundary, gate locking model, and object lifecycle
- Traced C/Go interop call chains through the **cgo** boundary using **gdb**
- Identified ARM64-specific behavior differences in CPython threading internals

Project Proposal

Abstract

The **go-mfp cpython package** is a unique Go library that embeds CPython as a scripting engine. It supports multiple isolated **sub-interpreters** within the same process, uses the host Python installation for version tolerance, and automatically garbage-collects Python objects on the Go side — eliminating manual memory management.

This package is planned to move into a **standalone repository** — similar to go-avahi — and could become **critical Linux infrastructure** if widely adopted as a generic embedded scripting engine. At that stage, the absence of serious bugs and vulnerabilities becomes essential.

This project proposes to **fuzz and test the cpython package following the same approach used for go-avahi** — writing unit tests (**_test.go*) and fuzz tests (**_fuzz_test.go*) for each source file, covering all public API functions across **Python 3.8 through the latest available version** on both x86 and ARM64.

Objectives

- **Audit** existing test coverage across all cpython package source files
- **Write unit tests** (**_test.go*) for each source file — mirroring the go-avahi pattern of pairing every *.go* file with a *_test.go* file
- **Write fuzz tests** (**_fuzz_test.go*) using Go's native *f.Fuzz()* framework — seeding with valid, invalid, and adversarial inputs and asserting invariants
- **Test across Python 3.8–latest** since cpython internals change significantly between minor versions
- **Test on both x86 and ARM64** — ARM64-specific issues are real, as demonstrated by the merged bug fix
- **Integrate with CI** for continuous automated testing across Python versions
- **Document** the test infrastructure and findings for future maintainers

Methodology

I will write unit tests (*_test.go*) and fuzz tests (*_fuzz_test.go*) for each source file in the cpython package, covering all public API functions across Python 3.8 through the latest available version.

Phase 1 — Coverage Audit and Unit Tests

- Run *go test -coverprofile* to generate a coverage report for all cpython source files
- For each source file (*python.go*, *gate.go*, *object.go*, *cpython.c* etc.), identify untested public functions
- Write unit tests covering: normal operation, boundary conditions, error propagation, and sub-interpreter isolation
- Validate tests pass on **Python 3.8, 3.10, 3.12, 3.13, 3.14, 3.15** using pyenv or Docker

Phase 2 — Fuzz Test Development

Following the go-avahi fuzz test pattern, I will write fuzz tests targeting:

- **Object creation and type conversion:** Fuzz `py.Eval()` and `py.Exec()` with malformed scripts, unexpected types, and adversarial inputs — asserting no panics and correct error returns
- **Sub-interpreter lifecycle:** Stress `NewPython()` / `Close()` cycles under concurrent access — asserting no deadlocks or memory corruption
- **Python object GC:** Fuzz object creation and abandonment patterns to verify automatic GC correctness
- **Error handling paths:** Fuzz Python exception scenarios to verify errors propagate correctly to Go without crashes
- **Cross-version invariants:** Run all fuzz tests against Python 3.8–latest to catch version-specific regressions — especially on ARM64 where threading behavior differs

Phase 3 — CI Integration

- Set up **GitHub Actions** matrix builds testing across Python 3.8–latest on x86 and ARM64
- Run fuzz tests in short-fuzz mode on every PR to catch regressions early
- Document how to run full fuzz campaigns locally and how to add new test cases
- Report and fix any bugs found during fuzzing

Tentative Timeline & Deliverables

Phase	Dates	Deliverables
Community Bonding	May 8 – Jun 1	Study go-avahi test structure in depth. Complete coverage audit of cpython package. Align test strategy with Alexander. Multi-version Python environment configured.
Phase 1 (Week 1–3)	Jun 2 – Jun 22	Unit tests written for python.go, gate.go, object.go. All tests passing on Python 3.12. Coverage report showing improvement.
Phase 1 (Week 4–6)	Jun 23 – Jul 13	Unit tests validated across Python 3.8–3.15 on x86. ARM64-specific test cases added. Any version regressions identified and reported.
Mid-Term Eval	Jul 14 – Jul 18	Complete unit test suite merged. Mid-term report submitted.
Phase 2 (Week 7–9)	Jul 19 – Aug 8	Fuzz tests written for object creation, sub-interpreter lifecycle, GC, and error handling. Initial fuzz runs completed on x86 and ARM64.
Phase 2 (Week 10–12)	Aug 9 – Aug 24	CI matrix builds across Python versions and architectures. Any bugs found during fuzzing fixed and verified. Documentation complete.
Final Week	Aug 25 – Sep 1	End-to-end validation. Final report submitted. All code and docs merged.

Availability

I am fully committed to GSoC 2026 and will dedicate 30+ hours per week throughout the program. I have no exams or other commitments during the GSoC period (May–August). I will communicate proactively with mentors via Telegram and GitHub and maintain consistent progress.

Post GSoC

I plan to continue maintaining the **cpython** test suite as the package moves to its standalone repository. As the package potentially becomes part of critical Linux infrastructure, I am committed to keeping the test suite current with new Python versions and contributing fixes for any regressions that emerge.